

Rapport de brief

Créer et maintenir un DataLake : IoT industriel

Contexte du projet

En tant que data engineer, concevoir l'architecture d'un data lake, déployer l'infrastructure de stockage objet, mettre en place les pipelines d'ingestion, cataloguer les données et implémenter les règles de gouvernance et de contrôle d'accès.

Leurs données sont aujourd'hui stockées en vrac, sans structure ni gouvernance. La DSI vous confie la mission de concevoir et déployer un data lake moderne pour centraliser, documenter et sécuriser l'ensemble de ces flux, en vue d'un futur projet de maintenance prédictive.

Ce client collecte en continu des données issues de plusieurs lignes de production instrumentées. Ces données sont actuellement dispersées et hétérogènes, ce qui complique leur exploitation et limite leur valeur métier. La mise en place d'un datalake permet de centraliser l'ensemble des données dans une plateforme unique, capable de stocker aussi bien les données brutes que les données transformées. Une organisation structurée des données facilite leur découverte, leur traçabilité, leur sécurisation et leur réutilisation par les différentes parties prenantes.

1. Architecture

Le projet organise les données en quatre couches successives : raw → staging → curated → archive, stockées sur MinIO (stockage S3). Un jeu de données de capteurs industriels (5 lignes de production) sert de source. Airflow orchestre le pipeline et OpenMetadata assure le catalogage et le suivi de lineage entre les buckets.

Les services sont packagés avec Docker Compose : MinIO (stockage), un conteneur mc éphémère pour la création des buckets, KES pour la gestion des clés de chiffrement, MySQL comme base partagée (schémas OpenMetadata et Airflow), Elasticsearch pour l'indexation de recherche, et le serveur OpenMetadata lui-même.

2. Intérêt des outils choisis

MinIO

MinIO offre un stockage objet compatible avec l'API S3, ce qui permet de reproduire fidèlement en local le comportement d'un data lake cloud sans dépendre d'AWS.. Cette compatibilité API facilite aussi une éventuelle migration, le code applicatif (boto3) restant identique.

Airflow

Airflow permet d'orchestrer des pipelines de données sous forme de DAGs, avec gestion des dépendances, des reprises sur erreur, de la planification et de la génération dynamique de tâches. Il apporte une visibilité claire sur l'état de chaque étape du pipeline et facilite la maintenance par rapport à des scripts exécutés manuellement ou par de simples tâches cron.

OpenMetadata

OpenMetadata joue le rôle de catalogue de données central : il référence les schémas des fichiers, documente les colonnes, et surtout trace le lineage entre les différentes étapes du pipeline. Il permet de savoir d'où vient une donnée, quelles transformations elle a subies, et qui peut y accéder.

3. Pourquoi ces fonctionnalités ont été ajoutées

Upload par chunks

Le script `to_raw.py` importe la données par chunks dès que le fichier dépasse une taille de part configurable (8 Mo par défaut). L'intérêt est double : d'une part, cela évite de charger un fichier volumineux entièrement en mémoire avant l'envoi et cela rend l'upload plus robuste, puisqu'en cas d'échec sur une partie du transfert, seule cette partie doit être renvoyée plutôt que le fichier complet. C'est une bonne pratique standard dès qu'on manipule des fichiers de taille significative.

Trois comptes de service MinIO

Trois comptes distincts ont été créés pour appliquer le principe du moindre privilège :

Compte	Policy	Droits
svc-data-analyst	data-analyst-readonly.json	Lecture seule sur curated
svc-data-engineer	data-engineer-readwrite.json	Lecture et écriture sur raw, staging, curated
svc-admin	x	Pleins droits

Séparer ces rôles limite les risques : un data analyst n'a par exemple aucun moyen d'écrire ou de supprimer des données dans curated/, même par erreur, tandis que le compte admin reste réservé aux opérations d'infrastructure. Cela prépare le terrain pour une ouverture du data lake à plusieurs utilisateurs ou services sans risque de désastre en écriture. C'est le principe du moindre privilège.

Chiffrement des données (SSE-S3 + KES)

Chaque bucket est chiffré (SSE-S3) dès sa création, avec les clés fournies par KES . L'objectif est de garantir que les données stockées sur disque restent protégées même en cas d'accès non autorisé au système de fichiers sous-jacent — une exigence de base dès qu'on manipule des données industrielles ou sensibles.

Archivage automatique après 180 jours

Le DAG `minio_archive_180d` déplace quotidiennement vers `archive/` tout objet de `curated/` dont la date de dernière modification dépasse 180 jours. Cette fonctionnalité répond à un besoin classique de cycle de vie des données : conserver les données récentes dans une couche facilement accessible et peu coûteuse à interroger, tout en déplaçant les données anciennes vers une couche moins active, réduisant les coûts de stockage actif et gardant `curated/` pertinent pour les usages courants. Le mouvement est aussi tracé dans le lineage OpenMetadata, ce qui permet de conserver une trace complète du cycle de vie de chaque donnée.

4. Fonctionnement des DAGs Airflow

DAG de pipeline (`dag_pipeline.py`)

Un DAG est généré dynamiquement pour chaque fichier CSV trouvé dans `source/` (nommé `Pipeline_<nom_du_fichier>`, exécution quotidienne, sans rattrapage). Chaque DAG exécute trois tâches séquentielles :

- `import_to_raw` : upload du CSV source vers le bucket raw.
- `transform_to_staging` : téléchargement depuis raw, harmonisation des noms de colonnes (mise en minuscule), réupload vers staging, puis enregistrement du lineage.
- `transform_to_curated` : téléchargement depuis staging, réupload vers curated. La transformation elle-même est aujourd'hui un placeholder, mais le lineage staging - curated est tout de même enregistré.

DAG de maintenance (`dag_archivage.py`)

Voir “Archivage automatique après 180 jours” partie 3.

DAGs générés par OpenMetadata

Deux DAGs supplémentaires (`airflow_local_metadata.py` et `minio_metadata_extraction.py`) sont auto-générés par OpenMetadata lui-même à partir de fichiers de configuration JSON. L'un scanne les métadonnées d'Airflow, l'autre scanne le service de stockage MinIO pour en extraire les schémas.

5. Difficultés rencontrées

Déploiement d'OpenMetadata

La mise en place d'OpenMetadata s'est révélée longue et instable. OpenMetadata s'appuie sur plusieurs composants dépendants (MySQL, Elasticsearch, un job de migration de schéma exécuté une seule fois) qui doivent démarrer et s'initialiser dans le bon ordre avant que le serveur ne soit pleinement opérationnel, ce qui rend le déploiement plus fragile qu'un simple service isolé.

Cohabitation Airflow / MinIO / OpenMetadata sous Docker

La principale source de crash a été la cohabitation de deux instances Airflow tournant simultanément : le service ingestion défini dans le docker-compose du projet, et une instance Airflow gérée en interne par OpenMetadata (via son paramètre `PIPELINE_SERVICE_CLIENT`, qui pointe vers `http://ingestion:8080` pour déclencher les DAGs d'extraction de métadonnées). Ces deux instances partagent la même base et le même exécuter, ce qui a généré des conflits d'état et des redémarrages en boucle. Ce type de problème est courant

dès qu'un outil de catalogage embarque son propre orchestrateur tout en devant piloter un orchestrateur externe déjà en place : il faut veiller à ce qu'une seule instance fasse réellement autorité sur l'exécution des DAGs.

Sous-dossiers MinIO pour la documentation des colonnes

OpenMetadata ne parvient à extraire le schéma d'un fichier via le manifest de conteneur que si ce fichier réside dans un sous-dossier du bucket, et non à sa racine. Cette contrainte, propre à la façon dont OpenMetadata scanne les storage services, a nécessité l'ajout d'un préfixe commun `lines/` devant chaque nom de fichier dans tous les buckets. Il s'agit d'un contournement technique plutôt que d'un choix d'architecture initial, mis en place uniquement pour que le catalogage des colonnes fonctionne correctement.

6. Limites connues

Archivage au niveau fichier, pas ligne à ligne

L'archivage à 180 jours agit sur des objets entiers : un fichier CSV complet bascule vers `archive/` dès qu'il dépasse l'âge de rétention, sans distinction entre les lignes qu'il contient. Il n'est donc pas possible d'archiver uniquement les enregistrements les plus anciens d'un fichier tout en gardant les plus récents dans `curated/`. Une historisation plus fine nécessiterait un découpage temporel des fichiers (par exemple par année/mois), qui n'est pas implémenté actuellement.

Chiffrement partiel

Le chiffrement au repos est bien activé sur les buckets (SSE-S3 via KES), mais les échanges entre Airflow et MinIO ne sont pas chiffrés en transit : la configuration actuelle utilise un endpoint MinIO en `http://` (`MINIO_SECURE=false`). Par ailleurs, KES est aujourd'hui déprécié côté MinIO — il n'est plus activement maintenu et a été remplacé par MinKMS pour la gamme AIS3. Il reste fonctionnel pour un usage local, mais devra être remplacé par une alternative maintenue avant tout usage au-delà du développement.

Lineage simpliste

Le lineage est aujourd'hui enregistré au niveau conteneur à conteneur (`bucket` → `bucket`), avec un rattachement optionnel au pipeline Airflow ayant effectué la transformation. Il ne descend pas au niveau du fichier individuel, ni a fortiori au niveau des colonnes. Une amélioration possible serait de tracer le lineage par fichier, voire par ligne de production, afin de permettre un suivi plus précis de la provenance de chaque donnée — un lineage au niveau colonne serait particulièrement utile pour auditer l'impact d'un changement de schéma.